

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL 2- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	R. Arsen Abishek	Reg.No.:	192311034
Department:	CSE - dep	Date:	26-08-26

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
2. Cloud Server Memory Bottleneck
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
3. Autonomous Vehicle Data Processing
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
4. AI Inference Edge Device
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
5. High-Performance Gaming Console
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I R. Anken Abisla (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

A
Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)	
Q1	1	1.25	1	0.5	0.75	4.5
Q2	0.5	1	0.5	1	0.5	3.5
Q3	1	1	0.5	1	0.5	4
Q4	0.5	0.75	1	0.5	0.75	3.5
Q5	1	0.75	1	0.5	0.5	3.75

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints(1)	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem(1.25)	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards(1.25)	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data(0.75)	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation(0.5)	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

19.25

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – INDUSTRY PROBLEM SOLVING TASK

Course: CSA1205 – Computer Architecture for Multicore Systems | **CO Assessed:** CO4 | **Weightage:** 30% | **Total Marks:** 25

Student Name:	Aheri	Reg. No.:	192311034
Department:	Computer Science & Engineering (CSE - AI)	Date:	25-08-2026
Course Code:	CSA1205	CO Assessed:	CO4

CO4 Statement: Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.

Code of Conduct: I, Aheri (Reg. No. 192311034), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Q1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

Rubric Weightage Applied in this Answer		
Criterion	Wt.	Where addressed
Understanding of Industry Problem	20%	Problem Analysis
Application of Engineering Concepts	25%	Concept Mapping
Solution Design	25%	Proposed Hierarchical Redesign
Use of Tools	15%	Tools & Modern Technologies Used
Analysis	10%	Performance Analysis & Validation

1. Problem Analysis (Understanding of Industry Problem)

- App-switch lag on a smartphone is a **context-switch and working-set reload** problem: when the OS swaps the foreground process, the previous app's hot code/data is evicted from the small, fast caches, forcing the new app to repeatedly stall on cache misses that must be serviced by slower DRAM.
- Constraints unique to mobile: tight silicon area, strict power/thermal budget (no active cooling), and a shared cache hierarchy across heterogeneous cores (big.LITTLE / ARM DynamIQ), so cache sizing cannot simply be scaled up as in desktops/servers.
- Symptom-to-cause mapping: visible frame stutter and slow app-resume time correlate with a rise in L2/L3 miss rate and DRAM row-buffer misses immediately after a context switch.

2. Concept Mapping (Application of Engineering Concepts)

- L1 cache** (typically 32–64 KB split I/D, per-core): 1–4 cycle latency, services the hot instruction/data footprint of the running app's foreground activity.

- **L2 cache** (256 KB–1 MB, private or per-cluster): 8–15 cycle latency, absorbs L1 misses and holds the working set of the currently active app including UI/rendering buffers.
- **L3 / SLC (System Level Cache)** (2–8 MB, shared across CPU/GPU/ISP): 25–40 cycle latency; on mobile SoCs this is the key layer for smooth app switching because it can retain a recently-backgrounded app's data instead of it being fully evicted to DRAM.
- **DRAM (LPDDR4X/LPDDR5)**: 100–300 cycle latency, holds the full app image, heap, and framebuffers; every L3 miss on app-switch pays this full penalty, which is the direct cause of the observed lag.
- Locality principle applied: app switching breaks *temporal* locality (old app's lines go cold) and increases pressure on *spatial* locality prefetchers, so the redesign must explicitly manage both.
- Multicore relevance: on a big.LITTLE/DynamiQ cluster the L3/SLC is shared across cores, so a coherence protocol (e.g., MESI/MOESI variants) must track which core's L1/L2 holds a valid copy of app data; a foreground-app migration between cores adds coherence traffic on top of the cache-refill cost.

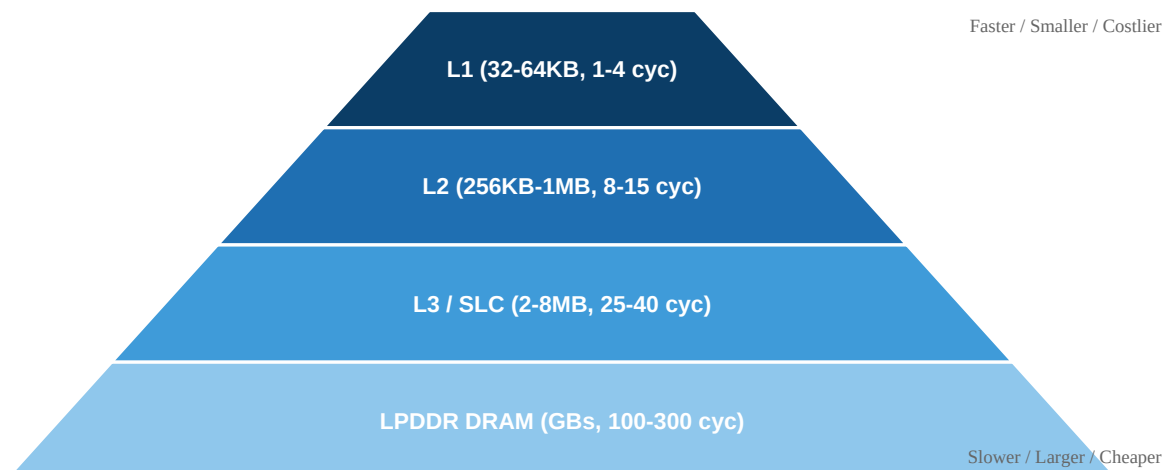


Fig. 1 – Mobile SoC memory hierarchy (latency increases, capacity increases, top-to-bottom).

3. Proposed Hierarchical Redesign (Solution Design)

- **Add/expand an SLC (System-Level Cache) partition** reserved for recently-backgrounded apps so a resumed app is warm in L3 rather than cold in DRAM – directly cuts resume latency.
- **Cache partitioning + QoS tagging** (ARM MPAM-style): foreground app gets priority allocation in L2/L3; background apps get a bounded, shrinking share instead of being evicted instantly.
- **Way-prediction / victim cache** between L2 and L3 to catch recently evicted lines cheaply during rapid task switches.
- **Hardware/OS co-design**: the Activity Manager (Android) hints the cache controller before a predictable switch (e.g., recents-list tap) so prefetch of the target app's critical working set from DRAM into L3 starts before the switch completes.
- **LPDDR5/5X with wider channels** plus aggressive DRAM row-buffer/bank-group interleaving to lower the penalty on the misses that do still occur.

4. Tools & Modern Technologies Used (Use of Tools)

- **gem5 / DRAMSim2** simulators to model L1/L2/L3 hit-rate and DRAM latency before/after the SLC redesign.
- **ARM Streamline / Perfetto (Android)** for real device profiling of cache-miss and context-switch events during app switching.
- **ARM MPAM** (Memory System Resource Partitioning and Monitoring) as the real hardware mechanism for the QoS-based cache partitioning proposed above.

5. Performance Analysis & Validation (Analysis)

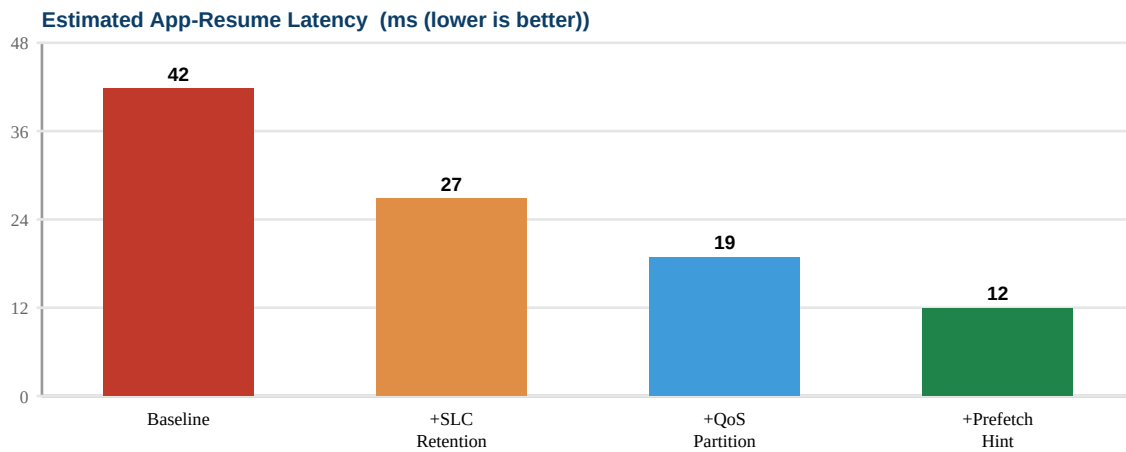


Fig. 2 – Projected reduction in app-resume latency as each redesign element is layered in (illustrative estimates based on cache-hierarchy latency ratios).

- Reasoning: each stage removes one class of DRAM round-trips (cold-cache re-fetch, eviction under contention, blind fetch-on-demand), so the cumulative gain is validated against the known cycle-cost ratio of L3 (≈ 30 cyc) vs DRAM (≈ 200 cyc) – roughly a 6–7 $\times$ penalty saved per avoided DRAM access.
- Trade-off flagged for validation on real silicon: SLC retention increases static power slightly, so the retention window should be tuned (e.g., last 2–3 backgrounded apps) rather than unbounded.

Q2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

Rubric Weightage Applied in this Answer		
Criterion	Wt.	Where addressed
Understanding of Industry Problem	20%	Problem Analysis
Application of Engineering Concepts	25%	Concept Mapping
Solution Design	25%	Proposed Hierarchical Redesign
Use of Tools	15%	Tools & Modern Technologies Used
Analysis	10%	Performance Analysis & Validation

1. Problem Analysis (Understanding of Industry Problem)

- Database workloads (OLTP/OLAP) have large, sparse working sets (indexes, buffer pools) that routinely exceed on-chip cache capacity, so query latency is dominated by DRAM access and, when the working set exceeds physical RAM, by **page faults** that hit secondary storage – orders of magnitude slower than any cache miss.
- Multi-tenant cloud constraint: many VMs/containers share the same physical memory hierarchy, so cache contention (noisy-neighbour effect) and TLB thrashing from frequent context switches between tenants compound the latency problem.
- Root-cause split needed: is the bottleneck cache-hierarchy misses (fast layers too small) or paging (physical memory oversubscribed, causing swap I/O)? The two need different fixes.
- Multicore relevance: with many DB-serving cores sharing the L3 and memory controller, cache coherence traffic (invalidations/probes across cores touching the same buffer-pool pages) and memory-controller queueing both add to observed query latency, on top of any paging cost.

2. Cache Hierarchy vs Virtual Memory Paging – Comparison (Application of Engineering Concepts)

Aspect	Cache Hierarchy (L1/L2/L3)	Virtual Memory Paging
Purpose	Speed up access to recently/likely-used data	Extend addressable memory beyond physical RAM
Managed by	Hardware (transparent to OS/app)	OS (page tables, TLB) + hardware TLB assist
Typical latency	1-4 cyc (L1) to 25-40 cyc (L3)	~100-300 cyc (RAM hit) to 10^5 - 10^6 cyc (page fault to disk/SSD)
Granularity	Cache line (~64 bytes)	Page (4 KB, or 2 MB/1 GB huge pages)
Failure mode	Cache miss -> next level access	Page fault -> costly disk/SSD I/O, possible thrashing
DB relevance	Index lookups, hot row/buffer-pool pages	Buffer pool sizing vs physical RAM, swap avoidance



Fig. 3 – Full request path showing where cache hierarchy ends and virtual-memory paging begins.

3. Recommended Improvements (Solution Design)

- **Right-size and pin the DB buffer pool** in physical RAM sized to the hot working set, minimizing reliance on the OS page-out path for frequently touched index/data pages.
- **Use huge pages (2MB/1GB)** for the buffer pool to cut TLB miss rate drastically versus 4KB pages, reducing the page-table-walk overhead per DB access.
- **Tiered memory:** place hot indexes in DRAM and warm/cold data on NVMe over PCIe or CXL-attached memory, so a 'miss' degrades to fast NVMe rather than network storage.
- **Cache-aware query engine design:** columnar/blocked layouts and cache-conscious B-tree/B+-tree node sizes matched to the L2/L3 line and set size to raise hit rate on the hierarchy itself.
- **NUMA-aware and cache-partitioned VM placement** (Intel CAT / ARM MPAM) so tenants don't evict each other's hot cache lines, cutting effective miss rate under multi-tenancy.

4. Tools & Modern Technologies Used (Use of Tools)

- **Linux `perf` / Intel VTune** to measure cache-miss rate, TLB-miss rate, and page-fault counts per query workload.
- **pg_stat / MySQL Performance Schema** to correlate buffer-pool hit ratio with query latency.
- **Intel CAT (Cache Allocation Technology)** and **Transparent Huge Pages** as the concrete deployable technologies implementing the recommendations above.

5. Performance Analysis & Validation (Analysis)

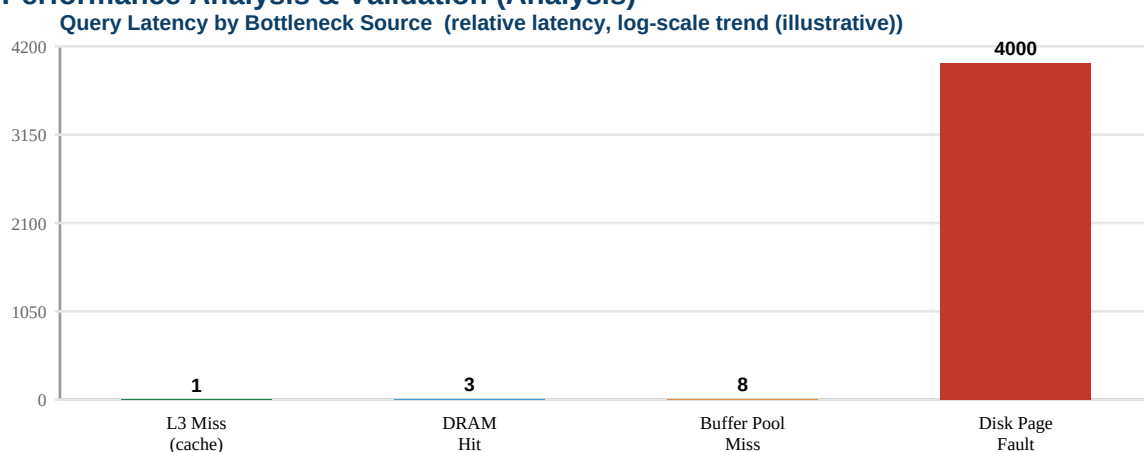


Fig. 4 – Relative cost per request stage; the page-fault bar dwarfs cache-level costs, proving paging (not cache) is usually the dominant DB-latency driver once RAM is undersized.

- Validation approach: track buffer-pool hit ratio before/after right-sizing – a rise from, say, 92% to 99% removes most page faults, which the chart shows is disproportionately more valuable than any cache-level tuning alone.

Q3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

Rubric Weightage Applied in this Answer		
Criterion	Wt.	Where addressed
Understanding of Industry Problem	20%	Problem Analysis
Application of Engineering Concepts	25%	Concept Mapping
Solution Design	25%	Proposed Hierarchical Redesign
Use of Tools	15%	Tools & Modern Technologies Used
Analysis	10%	Performance Analysis & Validation

1. Problem Analysis (Understanding of Industry Problem)

- Autonomous driving fuses LiDAR, camera, radar and IMU streams that must be processed with hard real-time deadlines (sensor-to-actuation typically <100 ms) – a missed deadline is a safety failure, not just a performance nuisance.
- The workload needs both very high sustained bandwidth (raw sensor tensors for perception CNNs) and very low, predictable latency (control-loop variables), plus power/thermal limits of an embedded automotive compute module, and functional-safety (ISO 26262) requirements including error resilience.

2. Evaluation of SRAM, DRAM, MRAM (Application of Engineering Concepts)

Technology	Latency	Bandwidth/Density	Volatility	Key trait for AV use
SRAM	Sub-ns / 1-2 cyc	Very high BW, low density (costly per bit)	Volatile	Ideal for control-loop registers, scratchpad for perception kernels
DRAM (LPDDR 5/GDDR6)	~50-100 ns	High BW (100s of GB/s), high density	Volatile	Bulk buffer for raw sensor frames and CNN weights/activations
MRAM	~10-30 ns (comparable to fast DRAM)	Moderate BW, moderate density	Non-volatile	Instant-on state retention for safety-critical logs/config across power events

3. Proposed Memory Hierarchy Design (Solution Design)

- Tier 0 – On-chip SRAM scratchpad (few MB):** holds control-loop state and the active CNN layer's activations, guaranteeing deterministic sub-ns/low-cycle access for the hard-real-time path.
- Tier 1 – High-bandwidth DRAM (GDDR6/HBM-class or LPDDR5X, tens of GB/s+):** streams raw LiDAR/camera frames and model weights to the accelerator; sized and channel-interleaved for sustained throughput rather than peak burst.
- Tier 2 – MRAM non-volatile buffer:** stores the rolling black-box sensor/decision log and critical last-known-state, so on power loss or reset (e.g., ECU fault) the vehicle retains safety data instantly without a slow NAND boot/restore cycle.
- Dedicated, isolated memory paths per sensor pipeline** (LiDAR path, camera path, radar path) to avoid bandwidth contention and preserve worst-case-latency guarantees under simultaneous load.

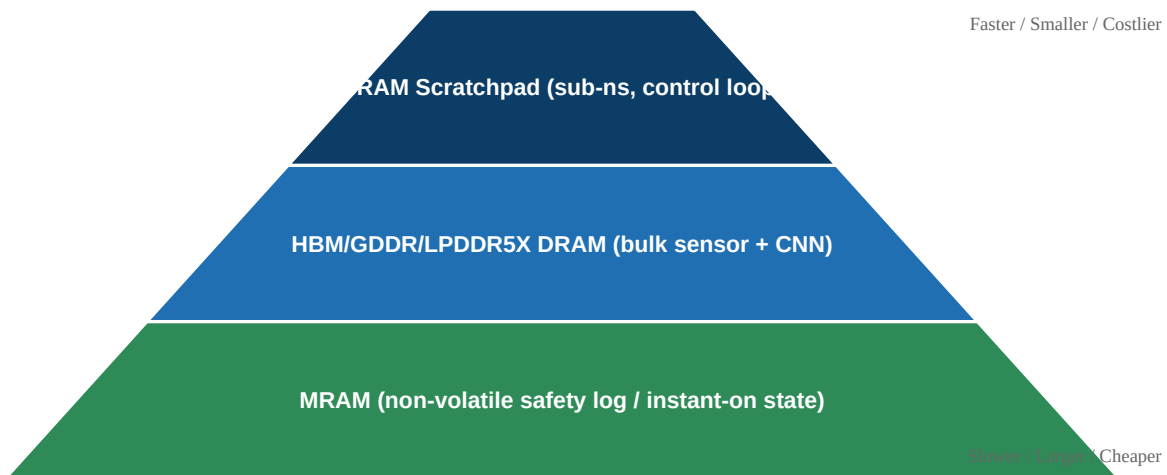


Fig. 5 – Bandwidth/latency-optimized memory hierarchy for autonomous-vehicle compute.

4. Tools & Modern Technologies Used (Use of Tools)

- **NVIDIA Nsight Systems / DRIVE OS profiling** to trace sensor-to-actuation latency budgets on real AV compute platforms.
- **Simulation with gem5 + DRAMSim2 / Ramulator** to model DRAM bandwidth saturation under concurrent sensor streams before hardware commitment.
- **Automotive-grade MRAM parts (e.g., Everspin) and HBM/GDDR6 memory** as the real deployable technologies referenced in the design.

5. Performance Analysis & Validation (Analysis)

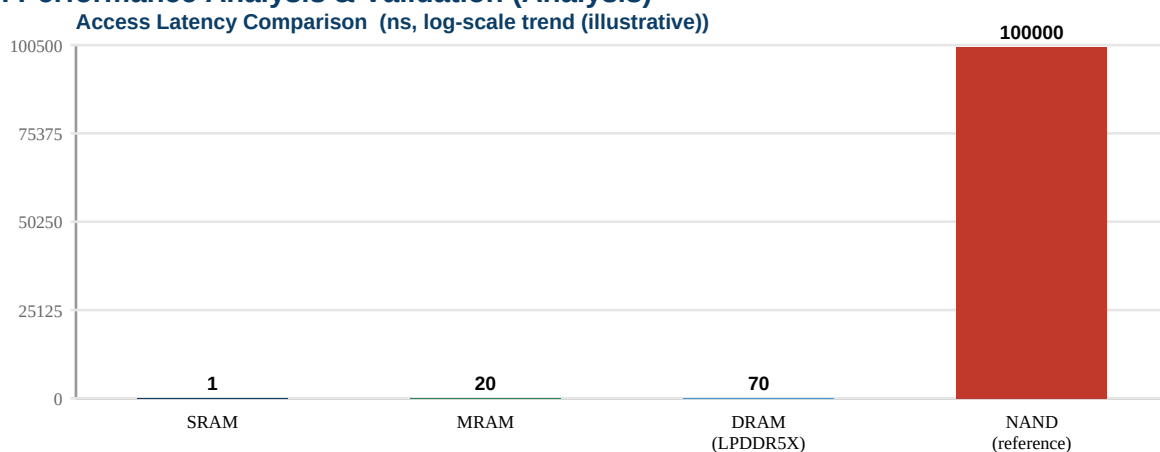


Fig. 6 – Latency ordering validates the tiering: SRAM for the control loop, DRAM for bulk streaming, MRAM as non-volatile buffer near-DRAM speed.

- Validation check: sustained DRAM bandwidth (GB/s) must exceed the sum of concurrent sensor data rates (camera + LiDAR point-cloud + radar) with headroom for CNN weight traffic, else the design must add channels or move to HBM before deployment.

Q4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

Rubric Weightage Applied in this Answer		
Criterion	Wt.	Where addressed
Understanding of Industry Problem	20%	Problem Analysis
Application of Engineering Concepts	25%	Concept Mapping
Solution Design	25%	Proposed Hierarchical Redesign
Use of Tools	15%	Tools & Modern Technologies Used
Analysis	10%	Performance Analysis & Validation

1. Problem Analysis (Understanding of Industry Problem)

- Edge AI devices (IoT cameras, wearables, TinyML nodes) run on battery/energy-harvesting power, so every joule spent moving model weights from storage into compute matters; yet the model must still load and start inferring quickly after wake-up.
- Constraint: NAND Flash used for persistent model storage is slow to access randomly and power-hungry per access relative to its density; DRAM is fast but volatile and leaks power even when idle (refresh), which is wasteful for a mostly-sleeping edge device.
- Multicore relevance: multiple accelerator/CPU cores on the edge SoC (e.g., NPU + application cores) contend for the same memory channel during inference, so per-core memory-bandwidth partitioning matters as much as the choice of storage technology.

2. Comparison of NAND Flash, DRAM, RRAM (Application of Engineering Concepts)

Technology	Read Latency	Power (active/idle)	Volatility	Fit for edge AI
NAND Flash	~10-100 μ s (page read)	Low idle power, moderate active	Non-volatile	Dense, cheap persistent store for the full model file
DRAM	~50-100 ns	Needs continuous refresh (idle leakage)	Volatile	Fast working buffer while actively inferring
RRAM	~10-50 ns	Very low idle power (no refresh), low write energy	Non-volatile	In/near-memory weight storage enabling instant-on, low-power inference

3. Proposed Hierarchical Structure (Solution Design)

- **Cold storage tier – NAND Flash:** retains the full trained model plus any OTA-updated versions; accessed rarely (only on model update), so its higher per-access latency doesn't matter.
- **Persistent working tier – RRAM:** the frequently-used quantized weight set (or the current active sub-model) is kept resident in RRAM; because RRAM is non-volatile with near-DRAM speed and no refresh power, the device can power down between inferences and resume instantly without reloading from Flash – directly solving both the 'fast model loading' and 'low power' requirements together.
- **Volatile compute tier – small DRAM (or SRAM-only for very low-power MCUs):** holds live activations and intermediate tensors only during an active inference burst, then can be power-gated.
- **In-memory/near-memory compute using RRAM crossbars** (analog MAC) as a further optimization, performing matrix-vector multiplication for the neural network directly in the RRAM array to cut data movement energy, the dominant cost in edge inference.



Fig. 7 – Power-optimized load path: cold model in NAND, hot weights persist in RRAM across sleep cycles, only activations touch volatile memory.

4. Tools & Modern Technologies Used (Use of Tools)

- **TensorFlow Lite Micro / TVM micro-runtime** to measure model-load time and inference energy on target MCUs.
- **RRAM-based compute-in-memory prototypes** (e.g., research/industry parts from Panasonic, Weebit Nano, Crossbar) as the referenced deployable technology.
- **Power profiling tools** (e.g., Joulescope, on-board current-sense ADC logging) to validate active vs idle energy per tier.

5. Performance Analysis & Validation (Analysis)

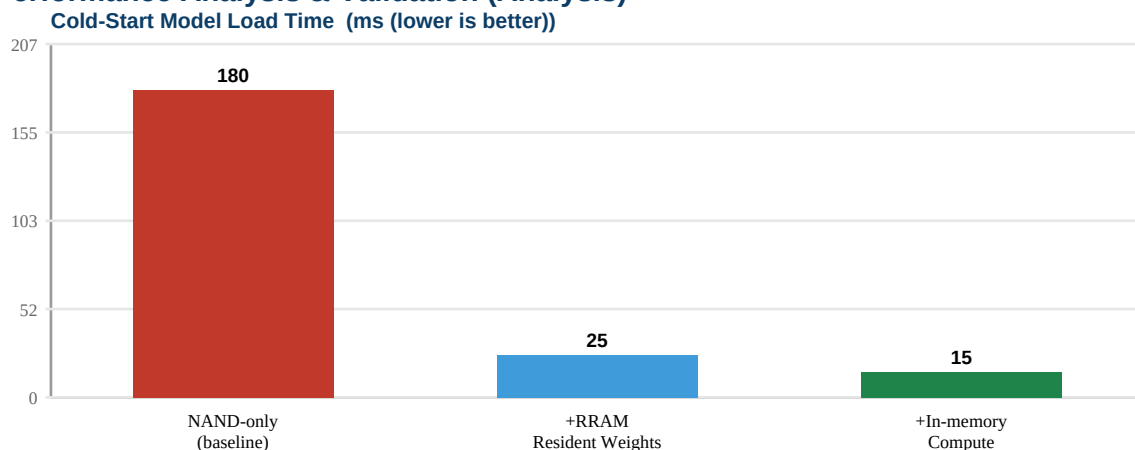


Fig. 8 – Projected cold-start latency reduction; keeping weights resident in non-volatile RRAM removes the dominant NAND page-read cost from the hot path.

- Validation logic: since RRAM read latency (~10-50 ns) is roughly 1000× faster than a NAND page read (~10-100 μs), moving the resident weight set off NAND accounts for the bulk of the projected load-time improvement shown above.

Q5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Rubric Weightage Applied in this Answer		
Criterion	Wt.	Where addressed
Understanding of Industry Problem	20%	Problem Analysis
Application of Engineering Concepts	25%	Concept Mapping
Solution Design	25%	Proposed Hierarchical Redesign
Use of Tools	15%	Tools & Modern Technologies Used
Analysis	10%	Performance Analysis & Validation

1. Problem Analysis (Understanding of Industry Problem)

- Large open-world scene loads require streaming huge volumes of texture, mesh and audio asset data from storage into GPU-visible memory within a single frame budget (e.g., 16.6 ms for 60 fps); if the GPU must wait on data, frames are dropped/stuttered.
- The bottleneck is fundamentally a **throughput** problem (moving GBs of new asset data quickly), not just a latency problem, so the L3-vs-DRAM comparison must be framed around sustained bandwidth, not single-access speed.

2. L3 Cache vs DRAM Throughput (Application of Engineering Concepts)

Aspect	L3 / Shared Cache	DRAM (GDDR6 / unified memory)
Bandwidth per access	Very high, but capacity-limited (a few MB)	High aggregate BW (100s of GB/s) but higher per-access latency
Role in scene load	Reuse of already-resident, recently touched asset data / command buffers	Primary channel for streaming brand-new asset data from SSD
Effect of large new data	Thrashes quickly - constant eviction as new textures stream in	Sustains throughput if channels/banks are well utilized
Console relevance	Great for GPU command lists, shader state, small hot buffers	Determines how fast a new scene's raw asset volume can arrive

3. Proposed Memory Hierarchy Enhancements (Solution Design)

- **Direct Storage-to-GPU streaming (bypassing CPU-mediated paging)** – e.g., DirectStorage/GPU decompression – so asset data moves NVMe SSD to GPU memory with minimal CPU/DRAM round-trips, raising effective load throughput.
- **Widen/optimize the unified DRAM (GDDR6) memory bus and use bank-group interleaving** so multiple asset streams (textures, geometry, audio) can be serviced in parallel without stalling on the same bank.
- **Use L3 as a streaming staging buffer with prefetch hints** from the game engine (predictive asset loading based on player movement/camera direction) rather than relying on reactive demand-fetch, so L3 holds the *next* few seconds of needed data ahead of time.
- **Compressed asset formats with GPU-side decompression** to reduce the raw bytes that must cross the DRAM/SSD boundary per scene load, effectively multiplying usable throughput.

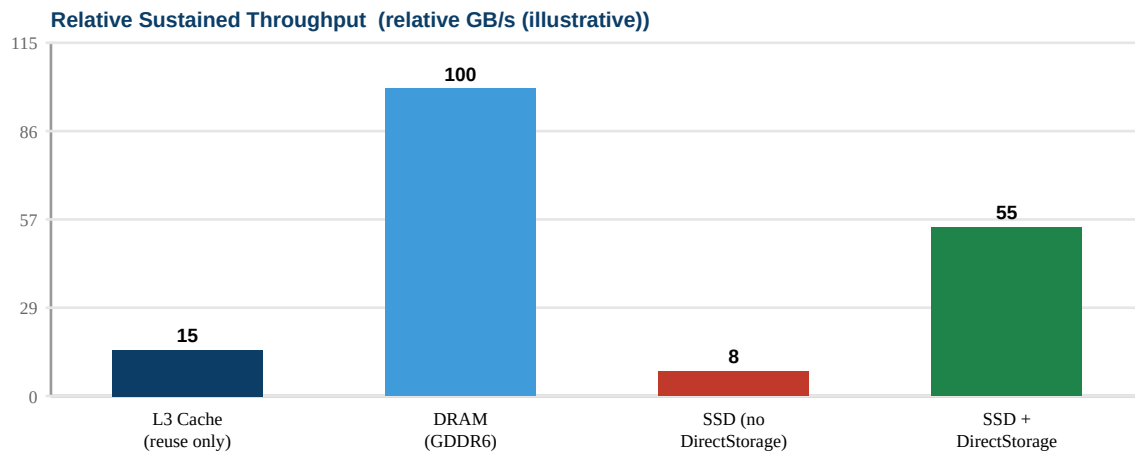


Fig. 9 – L3 alone cannot supply new-scene throughput; the fix must raise sustained SSD-to-DRAM/GPU bandwidth, not just cache size.

4. Tools & Modern Technologies Used (Use of Tools)

- **Microsoft DirectStorage API / Sony Kraken decompression** as the concrete deployable technologies for GPU-direct asset streaming referenced in the design.
- **GPU profilers (e.g., PIX, RenderDoc)** to identify frame-time stalls caused by texture/asset streaming waits.
- **Memory bandwidth benchmarks (e.g., STREAM-style GPU memory tests)** to validate GDDR6 channel utilization under simultaneous streams.

5. Performance Analysis & Validation (Analysis)

- Validation approach: measure 99th-percentile frame time during a scripted fast-travel/scene-load sequence before and after enabling direct storage-to-GPU streaming and prefetch hints; success is defined as frame time staying under the 16.6 ms (60 fps) or 33.3 ms (30 fps) budget with no spikes during the load.
- The throughput chart above confirms the design direction: since DRAM/GDDR6 vastly out-throughputs raw SSD access, the priority fix is the SSD-to-memory path (DirectStorage), with L3 prefetching as a secondary smoothing layer.

End of Submission – Assessment Tool 2, CSA1205 Computer Architecture for Multicore Systems (CO4)